Simulating the Cognitive Thought Process with a Modified Genetic Algorithm

Poojangi Ashok Gadekar
Department of Computer Engineering
MKSSS's Cummins College of Engineering for Women
Pune, India
gadekarpoojangi@gmail.com

Dr. Shubhangi Vinayak Tikhe
Department of Computer Engineering
MKSSS's Cummins College of Engineering for Women
Pune, India
shubhangi.tikhe@cumminscollege.in

2021 International Conference on Artificial Intelligence and Machine Vision (AIMV) | 978-1-6654-4211-4/21/$31.00 ©2021 IEEE | DOI: 10.1109/AIMV53313.2021.9670980

***Abstract*— Cognitive Computing is a developing paradigm with its applications found in almost every field. It aims to develop computing methodologies and systems inspired by mind's capabilities. A thought is the most fundamental capability of the mind. Hence it is important in the cognitive computing to understand the Thought Process. The purpose of this paper is to develop a computer model that simulates the Thought Process. The paper proposes that the Genetic Algorithm (GA) can be used for the same. Both cognitive model and computer model of the thought process with the help of GA has been given. In the computer model, a modification of GA has been implemented, which consists of a new crossover operator called Learning Crossover operator. The new crossover operator is not a replacement but is a supplement to the existing crossover operators. The GA is implemented over the Travelling Salesperson Problem (TSP), which is a classical NP problem and hence the algorithm is possible to be implemented on any other problem. The modification to the GA aims to improve the Thought Process Simulation. But it can also improve the performance of GA, when further explored.**

***Index Terms* —Cognitive Computing, Genetic Algorithm, Traveling Salesperson Problem, Thought Process, Natural Selection.**

I. Introduction

Cognitive Computing (CC) is an emerging paradigm of intelligent computing methodologies and systems based on cognitive informatics that implements computational intelligence by mimicking the mechanisms of the brain [1]. The Mind is capable of using the Brain to the most extent. Hence, Cognitive computing aims to develop a coherent, unified, universal mechanism inspired by the mind's capabilities [2].

The research in CC is advancing rapidly and can be categorized into the three categories: providing faculties of (1) Knowledge (2) Thinking and (3) Feelings [3]. However, it is noteworthy that even though the faculties of knowing, thinking, and feeling are interconnected, there is a little work performed on the integral cognitive architectures that take all the three into account [3].

When considered these three fields together, *Thoughts* can be seen as the basis of all. Thinking is the chain of thoughts, knowledge comes by learning from thoughts, whereas feelings are expressed with thoughts. Thus, it is important in Cognitive Computing to simulate the Thought Process. Thought should serve as a basic unit of analysis [4].

The human thinking can be classified as human general thinking and human control thinking. Ideas, results, conclusions, deductions come from the Human General Thinking [5]. This paper refers the Human General thinking to as the Cognitive Thought Process.

The paper proposes an approach to simulate the cognitive thought process. The approach can be considered as a basic framework. It proposes that the Genetic Algorithm (GA) can simulate the thoughts to a basic level and can be modified to simulate with more accuracy. For that, a new Crossover Operator is proposed, which is not a replacement to the existing Crossover Operators but can be used along with the existing ones. The paper uses typical Travelling Salesperson Problem (TSP) problem as the research object to understand the feasibility of the proposed system.

The objective of the paper is not to improve the performance of Genetic Algorithm, but to propose: (1) a perspective of Genetic Algorithm to simulate the thought process, and (2) a modified Genetic Algorithm, with a new Crossover Operator called *Learning Crossover Operator*, to simulate thought process more accurately. But when further improved, the proposed *Learning Crossover Operator* can also improve the performance of GA.

Section II explains the conceptual cognitive model of the Thought Process with GA. Section III gives a basic implementation of GA. Section IV explains the modification that can be applied over the basic GA implementation. Both GAs in the Section II and III are implemented over TSP. Section V reviews the test results of the basic and the modified GA. It also compares the modified GA with an existing GA. Finally, section VI gives the Conclusion and Future scope.

II. Using GA to depict the mind's thought process

The Genetic Algorithm has global search capability and is used in the optimization problems. The paper proposes that GA can be used in tracking the mind's thought traversal. GA is an iterative and stochastic process that operates on a set of individuals (on a population), where each individual is identified by a gene that represents a solution to given problem. In case of TSP, a gene can contain one possible route covering all cities. Initially, these individuals are generated either randomly or based on some criteria. Then during each iteration, fitter individuals are selected and by performing GA operations on them, new individuals (offspring) are produced which are placed in next generation. At the end, when convergence is reached, the fittest individual is considered as final solution.

The human mind traverses the thoughts similarly, for e.g., finding a solution to the TSP manually. By saying 'traverses' it means a random search and not a proper finding based on any algorithm. While finding a solution, the mind initially generates random solutions. Then it uses them – either mixes

them, to create new solutions (GA Crossover operation) or creates some random solutions by modifying them (GA Mutation operation). It keeps iterating this either till there are no better solutions possible (GA Convergence Concept) or till some sufficient number of iterations.

Thus, GA operations are useful in creating new thoughts. A new thought is created with one of the following ways – (1) Randomly creating a thought based on mind's state, priorities, etc. - Initialization (2) Linking from previous thoughts - Mutation (3) Concluding, inferring from two or more thoughts - Crossover (4) Carrying forward previously hold thoughts - Elitism.

Mutation and Crossover are the processes of traversing from previous thought to a new thought. When a thought is generated, it's context cloud is created in our mind as shown in the Fig. 1. The questions like Why, When, How, etc. are the contexts of the thought. This context cloud is mostly used to create a new thought. A gene representation can consist of the thought and its context.

In GA, Mutation is simply changing the bits of a gene. Here we can further modify the concept. A new thought can be created from the context bits of gene of previous thought. From the multiple contexts, the one with the highest priority (fitness) can be chosen to create the new thought. This is essentially linking from previous thought. The process is explained in the Fig.2.

In case of concluding from multiple previous thoughts, we overlap two or more contexts of the previous thoughts. This can be depicted through the Crossover. The process is explained in the Fig.3.

Wang [6] infers that the brain can be seen as a parallel system with multiple threads of thoughts, which operate in real-time where the thoughts are focused on /chosen based on attention. Similarly, a population in GA can be considered as a collection of thoughts. Which parents/thought-thread to choose depends on the fitness of the thoughts – decided by the Attention Engine Priority. And new thoughts can be created with the GA operations as mentioned previously.

With this basic similarity, GA is very suitable for mimicking the thought processes. But GA is more like a random search for the optimal solutions. And one of the differences between the mind's thought traversal and the random traversal is that the mind also uses logic to find solutions as it goes ahead. In other terms, as the mind generates more and more solutions, it also understands the logic or patterns that can lead it to the optimal solution more quickly, which is essentially the learning.

The feasibility of this theory is checked by implementing it over TSP.

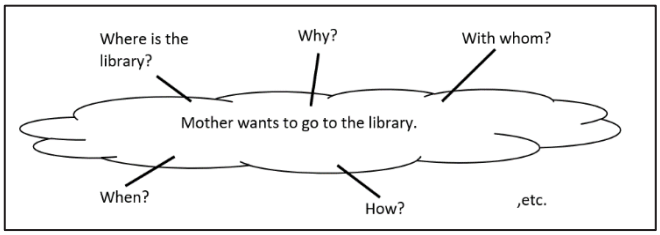

Fig 1. Context Cloud of a thought

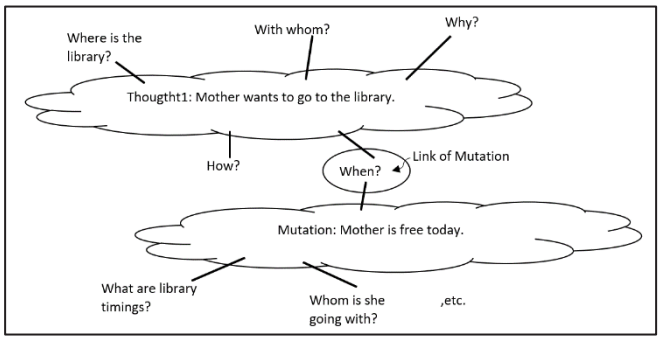

Fig 2. Mutation with the context cloud

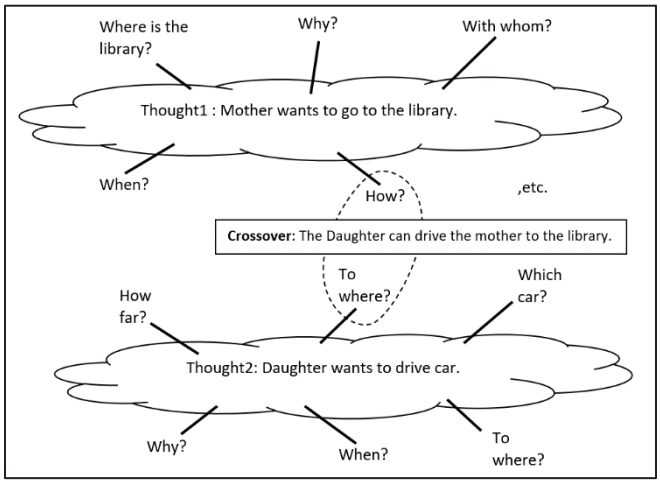

Fig 3. Crossover with the context cloud

III. Basic Implementation of GA over TSP

The basic implementation of solving TSP with GA is as shown in Fig.4. In basic GA, Selection Operator selects top elites. Crossover, Mutation, and Elitism produce offspring for the next generation. Crossover Operator creates offspring by mixing the genes of selected parents. Mutation Operator adds random individuals to the population, to increase diversity. Elitism helps sustain information i.e., top elites.

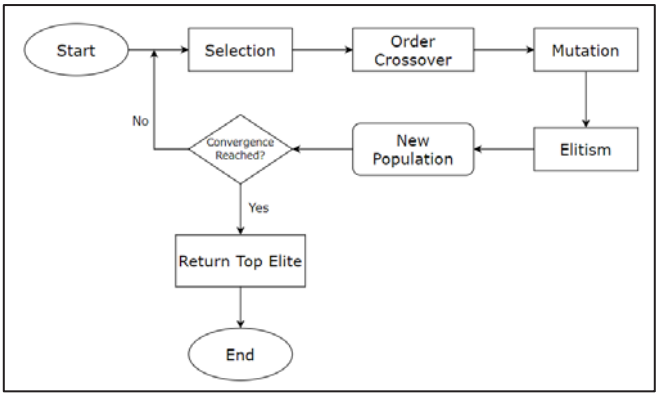

Fig 4. Basic GA implementation

A. Initialization and gene encoding

In TSP, there are different ways to represent/encode gene, like path representation, adjacency representation, ordinal representation, edge representation, etc. [7]

The paper uses common path representation. A route is regarded as a gene, where a route consists of all cities in a certain order. It is an array of integers that represent cities. Table I gives an example of a gene.

Table I. Gene representation

2	3	1	5	4	6	7	8

B. Fitness Calculation

The fitness for each individual decides how good is the solution proposed in its gene. In this paper, the fitness is calculated with eq.(1), where route_length is the total distance travelled with the given route.

$$fitness = \frac{1}{route\_length} \quad (1)$$

C. Selection

The selection is choosing two parents from a population for the process of crossing. The paper uses most common technique – Roulette Wheel Selection. Refer to Algorithm 1 for the illustration.

Algorithm 1 Roulette wheel Selection

```
Calculate sum of fitness of all individuals in the population as s.
for n times do
    Generate random number r from the interval (0, s)
    partial sum ⇐ 0
    for every individual in current population do
        partial sum ⇐ partial sum + individual fitness
        if partial sum > r then
            Select this individual as one of the parent.
            stop
        end if
    end for
end for
```

D. Crossover Operation

In case of TSP, the ordinary crossover can also generate invalid chromosomes (gene). For TSP, some common Crossover Operations that are modified to generate only valid chromosomes are Partially Mapped Crossover (PMX), Order Crossover (OX), Edge Recombination Crossover (ERX), etc. [8].

The paper uses Order Crossover, as it is more suitable for further modifications in the paper. The main Order Crossover is illustrated in Algorithm 2.

Algorithm 2 Order Crossover main

```
Make two random cuts c1 and c2.
For offspring1, offspring2 add cities between c1 and c2 from parent1 and
parent2, resp.
for remaining route do
    Call Order crossover from other parent(offspring1, parent2, indices)
    Call Order crossover from other parent(offspring2, parent1, indices)
end for
```

The Order crossover function makes use of the *Order_crossover_from_other_parent* function. This function is illustrated in Algorithm 3.

Algorithm 3 Order crossover from other parent

```
Input: offspring, parent, index1, index2
for route do
    if parent[index1] not in offspring then
        offspring[index2] ⇐ parent[index1]
    end if
end for
```

E. Mutation

In this paper, the Reverse Sequence Mutation (RSM) operator is used, because it is one of the most efficient algorithms for solving TSP [9]. The algorithm for Reverse Sequence mutation is as illustrated in Algorithm 4.

Algorithm 4 RSM Mutation on individual

```
select two random points p1 and p2.
while p1 < p2 do
    swap elements at positions p1 and p2.
    p1 ⇐ p1 + 1
    p2 ⇐ p2 − 1
end while
```

F. Elitism

Elitism can be used to eliminate the chance of any undesired loss of information during the mutation stage [7]. The first few best individuals of the current population are copied to the next population. The number of elites to be carried forward depends on the size of the problem.

IV. Modification to the Genetic Algorithm

As illustrated, GA depicts the way thoughts are traversed. What it lacks or what can be added to it is the learning factor.

The existing Crossover operators have a randomness factor. Hence there is no guarantee that the optimal solution will be reached in minimum iterations.

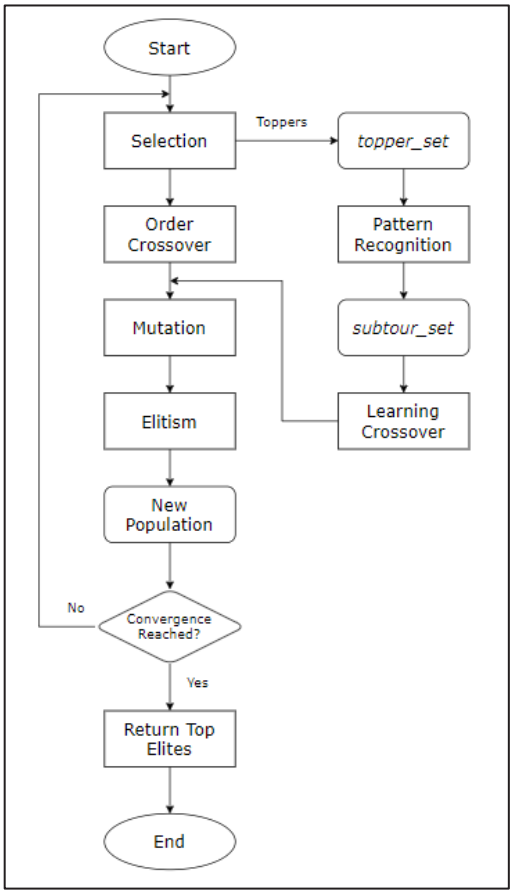

Fig. 5 Modified GA

The paper proposes a modification that is inspired by the machine learning technique, to enable the GA to learn and use the inferences in creating better solutions. The paper proposes a new supplementary Crossover technique.

The modification is illustrated in Fig. 5.

In Machine Learning (ML), training data is used to train the model. The model 'learns' from the data, by finding patterns in the data, like commonalities, differences, etc.

In GA, new individuals are created with crossover and mutation operators in each iteration. These individuals can be used as the data to train GA, their fitness being the labels as analogous to those in ML.

When the Selection operator selects top elites as parents, those elites are also stored into '*topper set'*.

Learning here will be extracting common subtours from these elites. Store those subtours in '*subtour_set'*. Use them to create new offspring in the new Crossover operator called **'Learning Crossover'**. The offspring generated by the Learning Crossover operator has more chances of being fitter. The modified GA is as illustrated in algorithm 5.

Algorithm 5 Modified GA

```
Maintain topper-set for top fit elites of all generations.
Maintain subtour-set for common subtours of all generations.
while Convergence not reached do
    perform Selection and store the output in selected-elites-for-iteration
    perform Crossover
    if elites in selected-elites-for-iteration not in topper-set then
        add elites in topper-set
    end if
    Extract common subtours between new elites.
    Extract common subtours of newly added elites with old elites in topper-set.
    if If extracted common subtours not in subtour-set then
        add in subtour-set
        Set learning-crossover-flag as True.
    end if
    if If learning-crossover-flag is True then
        Run Learning Crossover()
    end if
    perform Mutation
    perform Elitism
end while
```

The Learning Crossover operator uses a part of the Order Crossover operator logic. For an offspring, the common subtours from the *subtour_set* are used to fill its gene. Then for filling the remaining cells of the gene, a logic similar to the order crossover operator is used. The order of cities from both the parents is retained and thus two copies of the respective offspring are created. The offspring with the maximum fitness is selected to proceed further. The learning crossover logic is explained in the Fig.6. The algorithm for the Learning Crossover operator is as given in Algorithm 6.

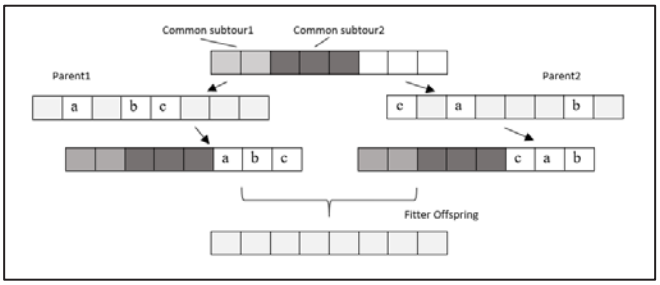

Fig 6. Learning Crossover Logic

Algorithm 6 Learning Crossover

Input: subtour-set

```
for each subtour in subtour-set if not already handled do
    Create an empty offspring as new-offspring
    fill subtour in new-offpsring
    for remaining bits in new-offspring do
        Offspring1 = Order-crossover-from-other-parent(parent1)
        Offspring2 = Order-crossover-from-other-parent(parent2)
        Select the one with higher fitness among Offspring1 and Offspring2 as the new-offspring
    end for
end for
```

V. EXPERIMENTAL RESULTS

The paper proposes a theory of simulation, which is a novel approach, and the author did not find any research work related to this to compare with. The theory is implemented over TSP, with the goal of verifying the feasibility of the theory. And it is found to be feasible, as the modified GA has been successfully implemented with its results being comparable.

The goal of this paper is not to propose a GA better than the existing ones. But the model has been tested to analyse the performance.

The modified GA performance is compared with that of the basic GA. These experimental results are as shown in Table II. The basic GA and the modified GA both are implemented in Python. Both programs were run on the following benchmark problems: FIVE, P01, G17 and Bays29. Both programs were run over 200 iterations and each case is repeated for 20 runs. The best distances found are given in Table II.

The modified GA is also compared with an existing GA over a benchmark problem Bayg29. For that purpose, the paper has referred the GA implementation from Fengrui[10] as the existing GA. The comparison is given in Table III.

It is found that modified GA works better over a small number of cities, but as the number of cities is increased, its performance seems to degrade. This behaviour is observed because, the subtours learned should be handled better when creating new offspring from them. The algorithm creates new offspring for each subtour, which leads to redundant offspring generation. The model has been implemented with very simple operators and is not enhanced to work well with large population. Hence there is performance degradation when n is increased. Creating disjoint sets of subtours and producing offspring from these disjoint sets can be implemented to solve this issue. Also, model can be improved to work better with large population size.

As the algorithm is seen not performing its best for large n, it is compared with an existing GA for n=29 i.e., a relatively smaller n. The modified GA is found to be performing better than the existing GA till n=29.

TABLE II. Experimental Results for TSP using basic GA and modified GA

Problem	No. of Cities	Best in 200 iterations		Known Optimal
		Basic GA	*Modified GA*	
FIVE	5	19	19	19
P01	15	348	304	291
Gr17	17	2311	2238	2085
Bays29	29	3028	3266	2020

TABLE III. Comparison of Modified GA with Existing GA over Bayg29

	Existing GA		Modified GA		Known Optimal
	Best	Mean	Best	Mean	
Distance	3122	3197	2271	2082	1610
Iteration	247	392	228	334	-

VI. CONCLUSION AND FUTURE SCOPE

The paper aimed to propose a basic, uncomplicated, and generalized model for the Cognitive Thought Process simulation. The model used a Genetic Algorithm with modification of a supplementary crossover operator i.e., Learning Crossover Operator. To understand it's feasibility, the model was implemented for simulating the thought process in solving the TSP and the results were found within our expectations.

The model is to be considered as a base framework for the thought simulation, and when to be implemented practically, the operators should be defined according to the application.

The concept of Natural Selection is found achievable and powerful in simulating the Thought Process.

Further work can be focused on improving the Learning Crossover operator with the objective of improving the performance of Genetic Algorithm. The future scope of the project can be defined over the practical operators of GA for the Thought Process Simulations in the three research pillars of Cognitive Computing – i.e., Thinking, Knowledge and Feelings. Also, the conceptual cognitive model of thought process can be further explored over the given perspective.

REFERENCES

[1] Y. Wang et al., "*Perspectives on Cognitive Informatics and Cognitive Computing: Summary of the Panel of IEEE ICCI'09*," 2009 8th IEEE International Conference on Cognitive Informatics, 2009, pp. 9-27, doi: 10.1109/COGINF.2009.5250815.

[2] By Dharmendra S. Modha, Rajagopal Ananthanarayanan, Steven K. Esser, Anthony Ndirango, Anthony J. Sherbondy, Raghavendra Singh *Communications of the ACM*, August 2011, Vol. 54 No. 8, Pages 62-71; 10.1145/1978542.1978559

[3] J. O. Gutierrez-Garcia and E. López-Neri, "*Cognitive Computing: A Brief Survey and Open Research Challenges*," 2015 3rd International Conference on Applied Computing and Information Technology/2nd International Conference on Computational Science and Intelligence, 2015, pp. 328-333, doi: 10.1109/ACIT-CSI.2015.64.

[4] Shadrikov, V., Kurginyan, S., Martynova, O. (2016) Psychological Studies of Thought: Thoughts about a Concept of Thought. *Psychology. Journal of Higher School of Economics*, *13*(3), 558-575, DOI: 10.17323/1813-8918-2016-3-558-575

[5] P. Wang, "*Learning and Control of Human-Like Thinking Process*", 2006 International Conference on Machine Learning and Cybernetics, 2006, pp. 850-854, doi: 10.1109/ICMLC.2006.258484.

[6] Y. Wang et al., "*The cognitive process and formal models of human attentions*," International Journal of Software Science and Computational Intelligence, vol. 5, no. 1, pp. 32-50, 2013.

[7] S.N.Sivanandam · S.N.Deepa, "*Introduction to Genetic Algorithms*", Springer-Verlag Berlin Heidelberg 2008, doi: 10.1007/978-3-540-73190-0

[8] Abid Hussain, Yousaf Shad Muhammad, M. Nauman Sajid, Ijaz Hussain, Alaa Mohamd Shoukry, Showkat Gani, "*Genetic Algorithm for Traveling Salesman Problem with Modified Cycle Crossover Operator*", Computational Intelligence and Neuroscience, vol. 2017, Article ID 7430125, 7 pages, 2017. https://doi.org/10.1155/2017/7430125

[9] Otman Abdoun, Jaafar Abouchabaka, Chakir Tajani, "*Analyzing the Performance of Mutation Operators to Solve the Travelling Salesman Problem*" arXiv:1203.3099 [cs.NE]

[10] F. Yu, X. Fu, H. Li and G. Dong, "Improved Roulette Wheel Selection-Based Genetic Algorithm for TSP," 2016 International Conference on Network and Information Systems for Computers (ICNISC), 2016, pp. 151-154, doi: 10.1109/ICNISC.2016.041.